



TOBB ETU University of Economics & Technology

Department of Computer Engineering

Senior Design Project

ANOMALY-BASED ACCIDENT DETECTION AND FAULTY VEHICLE
IDENTIFICATION SYSTEM (ADVIS)

Low-Level Design Report

Ahmet Faruk Sevim, Muhammet Emir Dizdar, Utku Çetin, Semih Uçan, Hasan Uzun

Supervisor: Assoc. Prof. Dr. Yücel Çimtay

Low-Level Design Report Feb 14, 2026

Table of Content

1. Introduction	2
1.1 Object design trade-offs	3
1.1.1 Usability vs. Functionality	3
1.1.2 Efficiency vs. Accuracy	3
1.1.3 Security vs. Usability	4
1.1.4 Reliability vs. Compatibility	4
1.2 Interface documentation guidelines	5
1.3 Engineering standards	6
1.4 Definitions, acronyms, and abbreviations	6
2. Packages	8
2.1 Perception and Tracking Package	8
2.2 Anomaly Analysis and Triggering Package	10
2.3 Diagnostic and Forensic Identification Package	12
2.4 Data Management and Reporting Package	13
3. Class Interfaces	14
3.1 Perception and Tracking Package Interfaces	14
3.2 Anomaly Analysis and Triggering Packages	16
3.3 Diagnostic and Forensic Identification Package Interfaces	19
3.4 Data Management and Reporting Package Interfaces	22
4. GLOSSARY	25
5. References	27

1.Introduction

Traffic safety is one of the most critical elements of modern urban life, and accidents affecting millions of people worldwide every year impose a significant burden on social welfare. Today, although cities are monitored by thousands of cameras, the detection of accident moments and response processes still largely depend on the attention of human operators or post-accident reports. This situation causes emergency response times to be prolonged and judicial processes to be hindered in identifying faulty vehicles after an accident. Therefore, transforming urban monitoring infrastructure from a passive recording device into a proactive intelligence that recognizes events instantly has become more important than ever.

The **Accident Detection and Faulty Vehicle Identification System (ADVIS)** is designed to detect traffic accidents in real-time by analyzing live video streams from city cameras using artificial intelligence and to automatically identify faulty vehicles in complex accident scenarios. The main purpose of ADVIS is to minimize human error by detecting accident anomalies within milliseconds and providing evidence-based reports to relevant authorities. The system uses object tracking and spatio-temporal analysis techniques to analyze not only a collision but also the irregular maneuvers that caused the accident. Furthermore, thanks to the system's "Triggered Diagnostic Pipeline" architecture, modules such as license plate recognition and accident classification, which require high processing power, are activated only when needed, ensuring maximum energy and performance efficiency on edge devices.

This report describes the low-level architecture and the detailed design of ADVIS. The report comprehensively covers the engineering trade-offs in object design, the international engineering standards on which the design is based, the logical package structure of the system, and the class interfaces. Finally, the document concludes with UML diagrams containing all the details of the system and detailed technical explanations of the software components.

1.1 Object design trade-offs

1.1.1 Usability vs. Functionality

In ADVIS, our primary objective is to deliver critical information with maximum clarity and minimal cognitive load for traffic operators. We believe that in emergency situations, "less is more." Therefore, we provide a clean, map-centric dashboard that allows users to understand the location and severity of an accident at first sight. While the system is capable of generating vast amounts of raw telemetry and metadata for every vehicle in the frame, we deliberately choose not to display all this functionality in the primary interface. Overloading the UI with secondary data would lead to complex, cluttered menus that could delay response times during a crisis.

To enhance usability, we have implemented an automated alert system that highlights only the most relevant "Solution Objects," such as the faulty vehicle and the incident clip, rather than requiring the user to manually filter through hours of footage. We avoid implementing redundant features or excessive configuration settings that would make the application difficult to quickly understand. By focusing on a "Triggered Visualization" approach, we ensure the system is as simple as possible for rapid decision-making, but not so simple that it lacks the essential evidence required for judicial processes.

1.1.2 Efficiency vs. Accuracy

The core objective of ADVIS is to process high-resolution video streams in real-time to detect accidents the moment they occur. However, analyzing multiple camera feeds simultaneously on a high-performance GPU-accelerated edge node presents significant computational challenges. Executing state-of-the-art deep learning models with maximum depth and ensemble techniques for every frame would provide the highest accuracy but would result in a massive drop in frames per second (FPS), making the system unusable for real-time safety.

For these reasons, we prioritize an efficient algorithm over a highly accurate but slow one during the continuous monitoring phase. We utilize lightweight model architectures

for the Perception Subsystem to ensure a high-frequency execution loop. By accepting a minor trade-off in pixel-perfect accuracy, we gain the efficiency required to maintain a seamless "Main Execution Loop." The system compensates for this by focusing on temporal consistency; even if a single frame's detection is slightly less accurate, the overall trajectory analysis remains robust enough to trigger the diagnostic branch where higher accuracy models can then be selectively applied.

1.1.3 Security vs. Usability

In ADVIS, protecting sensitive data such as license plate information and incident footage is a paramount concern, especially given the strict requirements of KVKK and GDPR. However, implementing extremely rigorous security layers, such as multi-factor authentication for every minor action or constant manual decryption processes, can severely hinder the usability of the system during a time-sensitive emergency. A traffic operator needs to access the accident data and faulty vehicle identity instantly to coordinate a response.

Therefore, we aim for a balanced approach where security does not become a barrier to rapid decision-making. We utilize automated AES-256 encryption for local storage and TLS 1.3 for data transmission, ensuring that the heavy lifting of security is handled in the background without affecting the user experience. While we maintain high security standards for data at rest and in transit, we provide an intuitive dashboard that allows authorized personnel to view incident reports without repetitive and cumbersome authentication steps during an active alert. We prioritize a seamless workflow that keeps the system "secure by design" but "accessible by necessity".

1.1.4 Reliability vs. Compatibility

The operational environment of ADVIS involves a wide variety of urban camera infrastructures, ranging from legacy analog systems to modern high-definition IP cameras. While ensuring the system is compatible with every possible video format and hardware interface is a desirable goal, excessive focus on broad compatibility can introduce software bloat and unpredictable bugs that undermine system reliability. In a

mission-critical application where an undetected accident could mean delayed medical intervention, reliability must take precedence over universal hardware support.

For these reasons, we prioritize a highly reliable system that is optimized for specific, high-standard data protocols and hardware platforms. Instead of aiming for universal compatibility with all legacy systems, which would require complex wrappers and potentially unstable drivers, we focus on standardized RTSP streams and NVIDIA-optimized processing pipelines. This trade-off allows us to ensure that the system remains stable and responsive under 24/7 operating conditions. By limiting the scope of compatibility to industry-standard interfaces, we can guarantee a higher degree of reliability and predictable performance, which is essential for a public safety system.

1.2 Interface documentation guidelines

In this report; all classes, attributes, and methods are named according to the standardized camel case convention to ensure consistency and readability across the system. Class names start with a capital letter (UpperCamelCase), while attributes and methods start with a lowercase letter (lowerCamelCase). The detailed technical specification for each class interface is presented in the following structured format:

class ClassName
This section provides a concise explanation of the class's primary responsibility within the system architecture, detailing its role in the overall workflow and its interactions with other components. <i>(Example Name: AccidentDetector)</i>
Attributes
This section lists the data structures and internal variables that define the object's state, specifying their data types and their significance in maintaining system data. <i>(Example Name: vehicleCount, isAnomalyDetected)</i>
Methods

This section specifies the functional behavior of the class, detailing the logic of each method, including its return types, input parameters, and the resulting state transitions.

(Example Name: calculateAnomalyScore(), updateTracker())

1.3 Engineering standards

We have used the UML guidelines in this report for the class interfaces, diagrams, scenarios, and subsystem compositions. UML is a globally recognized standard for generating these models; it is widely adopted for its clarity and ease of use in complex software projects, making it our primary choice for architectural visualization in the following pages. Furthermore, this report follows the IEEE standards for citations and technical documentation. As a commonly used and highly preferred method in the engineering community, adhering to IEEE standards ensures that our design descriptions remain professional, consistent, and compliant with industry-wide requirements.

1.4 Definitions, acronyms, and abbreviations

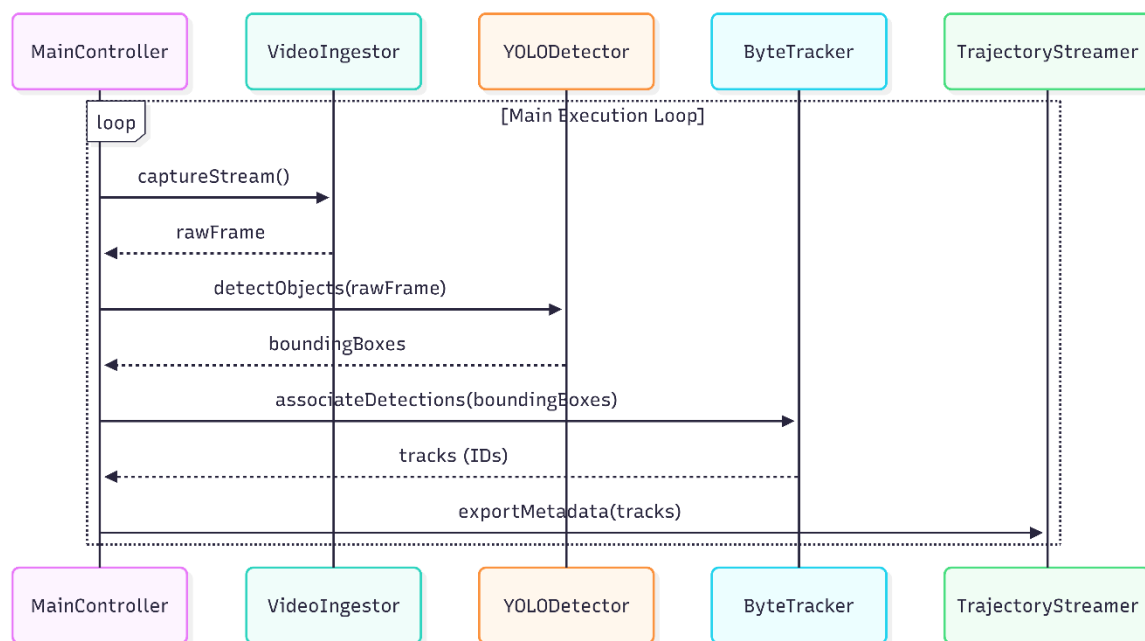
Term / Acronym	Definition
ADVIS	Accident Detection and Faulty Vehicle Identification System; the name of the project.
AOI	Area of Interest; specific geometric zones within the camera's field of view designated for monitoring.
CSI	Camera Serial Interface; a high-speed serial interface for connecting cameras to processors, often used in edge devices like Jetson.

GDPR / KVKK	General Data Protection Regulation and Kişisel Verilerin Korunması Kanunu; legal frameworks for data privacy and security.
HLD	High-Level Design; the architectural document that defines the system's subsystems and overall structure.
LLD	Low-Level Design; this document, which provides detailed technical specifications for classes, interfaces, and interactions.
LPR	License Plate Recognition; the technology used to identify vehicle identities from license plate images.
OCR	Optical Character Recognition; the process of converting images of text into machine-readable character strings.
RTSP	Real-Time Streaming Protocol; a network control protocol designed for use in entertainment and communications systems to control streaming media servers.
Spatio-Temporal	Analysis involving both spatial (location/space) and temporal (time) data points to detect behavioral patterns.
Tracklet	A short-duration fragment of a vehicle's trajectory consisting of associated detection boxes and a unique ID.
UML	Unified Modeling Language; a standardized modeling language used to visualize the design of a system.
YOLO	You Only Look Once; a state-of-the-art, real-time object detection system used for identifying vehicles in frames.

2. Packages

In this section, the ADVIS system is decomposed into logical software packages, refining the high-level subsystems identified in the HLD report. Each package represents a functional cluster of classes designed to work together to fulfill specific roles within the "Triggered Pipeline" architecture. This modular approach ensures that the high-frequency perception tasks are decoupled from the reactive diagnostic modules, enhancing both system maintainability and computational efficiency on edge hardware.

2.1 Perception and Tracking Package



This package is the entry point of the ADVIS pipeline, responsible for processing raw video input and maintaining continuous situational awareness. It transforms unstructured pixels into structured "Tracklets" by identifying and uniquely tagging every vehicle in the scene. This package corresponds to the "Main Execution Loop" that remains active indefinitely on the hardware accelerator.

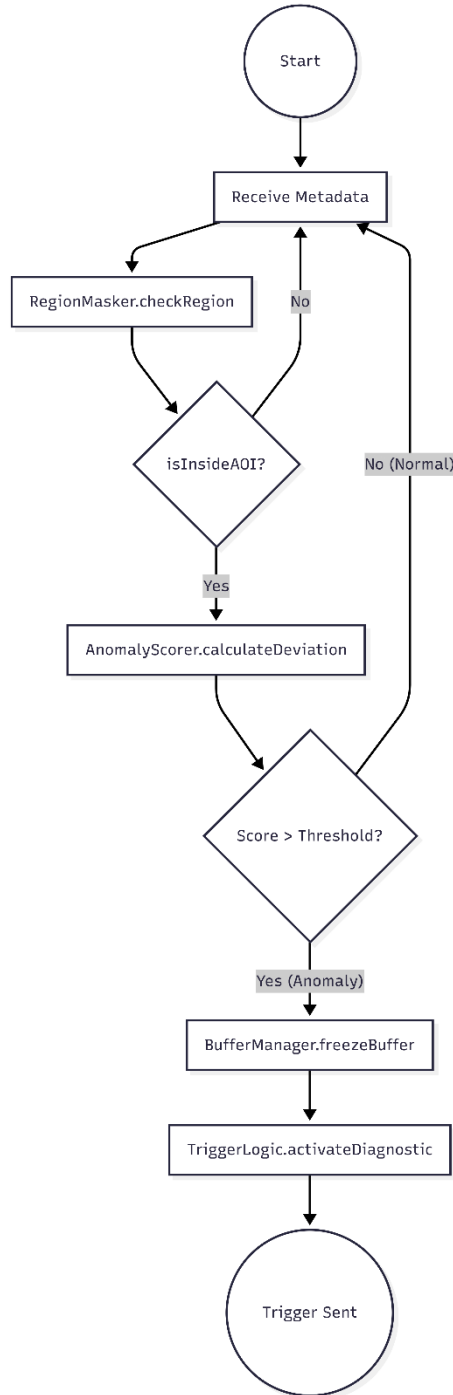
VideoIngestor: This class is responsible for capturing raw video streams (RTSP/CSI) and converting them into normalized frame buffers for processing. It acts as the primary producer in the system's asynchronous flow.

YOLODetector: This class implements the object detection service using lightweight YOLO-based models. It identifies bounding box coordinates and class labels for vehicles (cars, trucks, motorcycles) in real-time.

ByteTracker: This class provides the ID Management Service. By implementing the ByteTrack algorithm, it associates detections across consecutive frames to maintain temporal consistency and assign persistent unique identifiers to each vehicle.

TrajectoryStreamer: This class extracts and exports a continuous stream of motion metadata. It transforms tracking data into velocity vectors and (x, y) coordinates, which are required for the subsequent anomaly analysis.

2.2 Anomaly Analysis and Triggering Package



This package acts as the decision-making bridge between continuous perception and reactive diagnostics. It evaluates the structured metadata provided by the perception layer to detect irregular patterns in vehicle behavior. Once a statistical threshold is breached, this package generates an "Anomaly Trigger," which activates the high-fidelity

diagnostic branch of the system. This ensures that resource-intensive tasks, such as License Plate Recognition (LPR), are only executed when necessary.

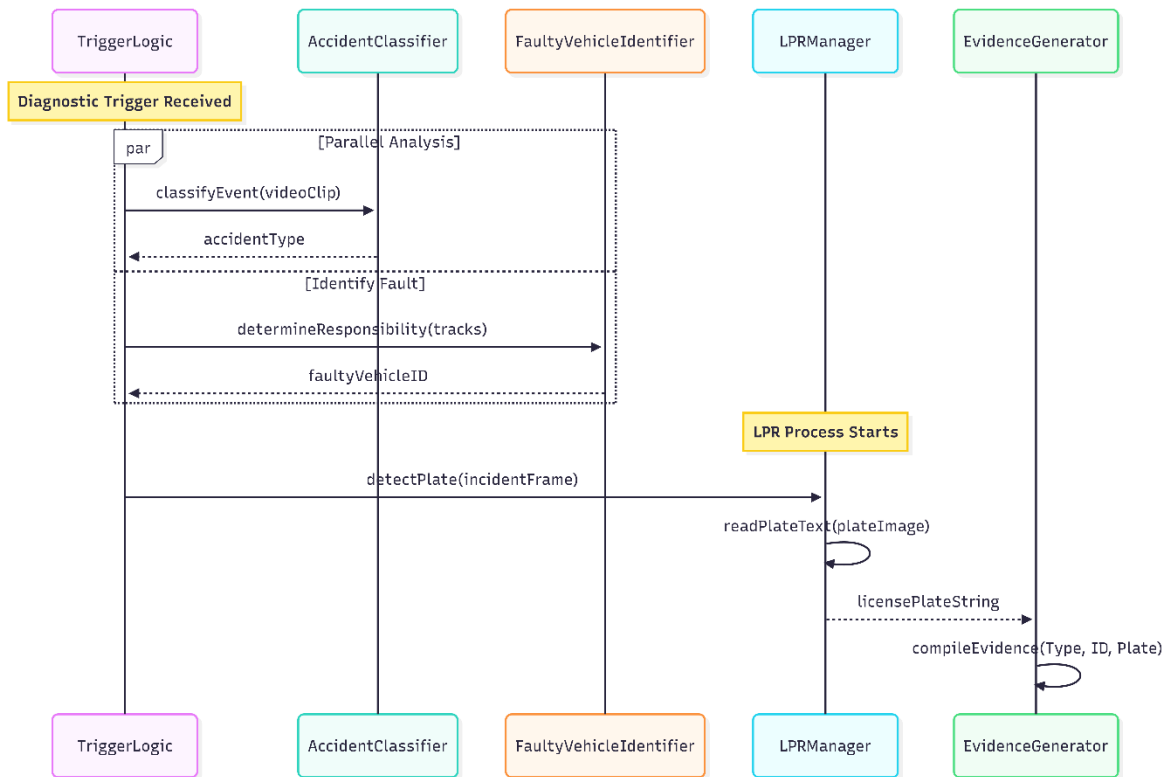
AnomalyScorer: This class implements the core spatio-temporal analysis algorithms. It calculates deviation scores for each tracklet based on sudden changes in velocity, trajectory, or proximity to other objects.

BufferManager: This class interacts with the perception layer's temporal storage to freeze the relevant "pre-event" video window. It ensures that the moments leading up to an anomaly are preserved for forensic analysis.

TriggerLogic: This class serves as the event-driven controller. It compares anomaly scores against pre-defined safety thresholds and manages the transition of the system state from "Observation" to "Active Diagnostic".

RegionMasker: This class defines static areas of interest (AOI) within the camera's field of view, allowing the system to ignore anomalies in non-critical zones, thereby reducing false alarm rates.

2.3 Diagnostic and Forensic Identification Package



This package is responsible for high-fidelity analysis once an "Anomaly Trigger" is received. It performs deep inspection of the frozen video buffer to classify the incident type and identify the involved vehicles through License Plate Recognition (LPR). This package operates reactively, ensuring that intensive computational resources are only utilized when a high-probability event is detected, maintaining the efficiency of the overall pipeline.

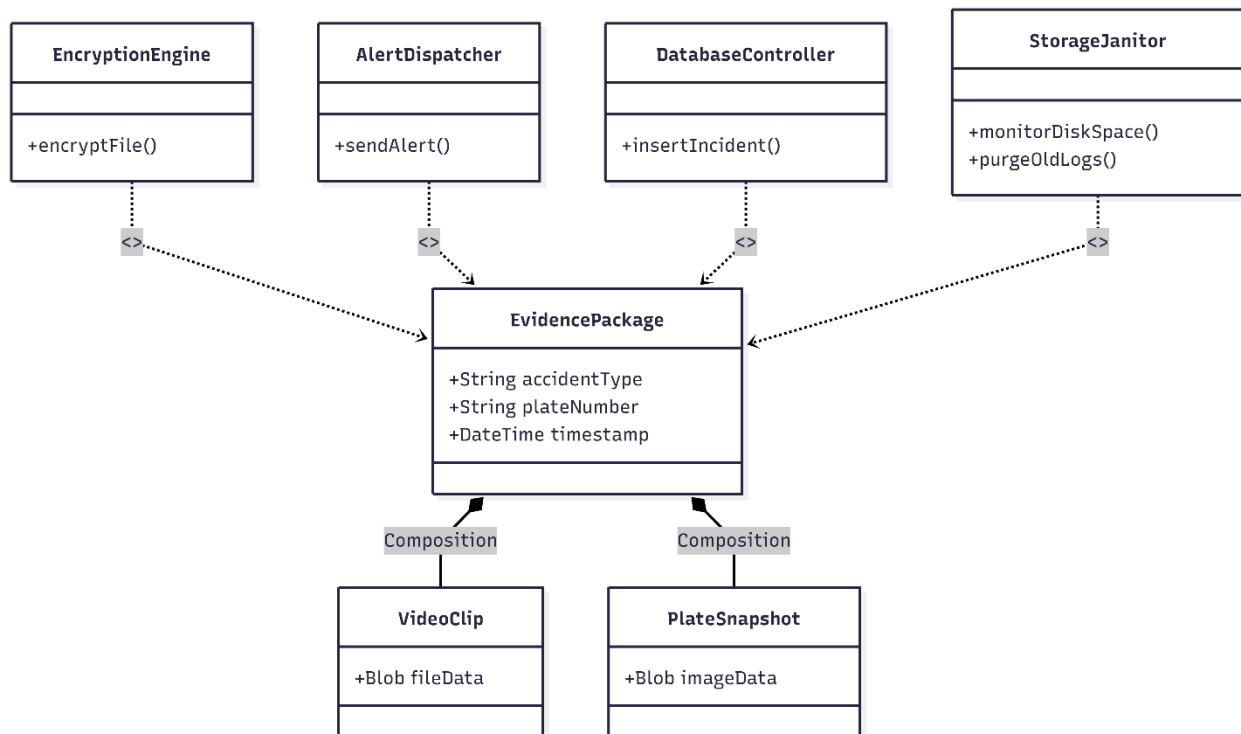
AccidentClassifier: This class uses specialized deep learning models to categorize the triggered event into specific incident types such as collisions, stalls, or illegal maneuvers. It analyzes the spatial relationship between tracklets during the anomaly window.

LPRManager: This class implements the License Plate Recognition service. It isolates the vehicle's license plate area from the high-resolution frames and utilizes Optical Character Recognition (OCR) to extract the alphanumeric identity of the involved parties.

FaultyVehicleIdentifier: This class applies rule-based logic to determine which vehicle is responsible for the detected anomaly. It correlates trajectory data with traffic laws (e.g., lane violations or sudden stops) to assign a "faulty" status.

EvidenceGenerator: This class aggregates the classification results, license plate data, and the specific video clip into a unified evidence package. It ensures that all diagnostic findings are ready for the reporting layer.

2.4 Data Management and Reporting Package



This package manages the final stage of the ADVIS pipeline, focusing on data persistence and alert dissemination. It ensures that all incident metadata and evidence clips are securely stored using AES-256 encryption to meet legal privacy requirements. Additionally, it provides the communication interface necessary to transmit high-priority alerts to the remote dashboard, enabling immediate response from traffic authorities.

DatabaseController: This class manages the interaction with the relational database (SQLite/PostgreSQL) to store incident metadata, such as time, location, and faulty vehicle IDs. It handles data indexing to allow for rapid forensic queries.

EncryptionEngine: This class implements the security protocols of the system. It is responsible for encrypting media files and sensitive identification data before they are written to the physical storage, ensuring compliance with data protection laws.

AlertDispatcher: This class acts as the notification hub. Upon receiving compiled evidence from the diagnostic layer, it formats the data into an HTTPS/TLS 1.3 message and pushes it to the remote client interface for real-time visualization.

StorageJanitor: This class manages the local storage lifecycle. It monitors disk space on the NVMe SSD and implements automated data retention policies, deleting the oldest non-incident footage to make room for new data while preserving all flagged evidence.

3. Class Interfaces

3.1 Perception and Tracking Package Interfaces

class VideoIngestor
Description: This class is responsible for capturing raw video streams from traffic cameras (via RTSP or CSI protocols) and converting them into normalized frame buffers. It serves as the primary data producer for the system.
Attributes
String streamSource: The network address or hardware port identifier of the camera.
CircularBuffer frameBuffer: A temporary storage holding the most recent 15-20 seconds of video frames for processing and potential event extraction.
Methods
void captureStream(): Establishes a connection with the camera source and initiates the continuous data stream.

Frame normalizeFrame(): Resizes and transforms the raw pixel data into the specific input format required by the detection model.

class YOLODetector

Description: This class implements the Object Detection Service. It utilizes lightweight YOLO-based deep learning models to identify and locate vehicles within individual video frames.

Attributes

Path modelWeights: The file path to the pre-trained neural network weights loaded into memory.

float confidenceThreshold: The minimum probability score required to classify a detected object as a valid vehicle.

Methods

BoundingBoxes detectObjects(Frame currentFrame): Processes a normalized frame to return the coordinates and class labels (e.g., car, truck, motorcycle) for all detected objects.

class ByteTracker

Description: This class provides the ID Management Service. By implementing the ByteTrack algorithm, it associates detections across consecutive frames to maintain temporal consistency and assign persistent unique identifiers.

Attributes

List<Track> activeTracks: A list of all vehicles currently being tracked within the camera's field of view.

int idCounter: An incremental counter used to assign a new unique ID to objects entering the scene.

Methods

Tracks associateDetections(BoundingBoxes bboxes): Matches new detection boxes with existing trajectories to update the state of active tracks and return updated tracking data.

class TrajectoryStreamer

Description: This class is responsible for the Trajectory Stream Service. It extracts structured motion metadata from tracking data and exports it for real-time anomaly analysis.

Attributes

Queue outputQueue: A synchronized queue containing structured motion data ready for the anomaly analysis subsystem.

Methods

VelocityVectors exportMetadata(Tracks tracks): Transforms raw tracking coordinates into velocity and direction vectors to be used as input for behavioral analysis.

3.2 Anomaly Analysis and Triggering Packages

class AnomalyScorer

Description: This class implements the core spatio-temporal analysis algorithms. It calculates deviation scores for each tracklet based on sudden changes in velocity, trajectory, or proximity to other objects.

Attributes

float spatialThreshold: The distance-based limit used to identify potential collisions or illegal maneuvers.

int temporalWindow: The time duration (in frames) over which motion patterns are analyzed to determine a behavioral trend.

Methods

float calculateDeviation(Tracks activeTracks): Evaluates current motion data against historical patterns to generate a numerical anomaly score.

List<int> detectCollisionIndices(): Identifies the specific vehicle IDs involved in a high-score event for targeted diagnostics.

class BufferManager

Description: This class manages the retrieval of video data from the perception layer's circular buffer. It is responsible for "freezing" the relevant time window once a trigger is received.

Attributes

int eventWindowSize: The total duration (e.g., 20 seconds) of video to be extracted, including pre-event and post-event footage.

Methods

VideoClip freezeBuffer(Timestamp triggerTime): Extracts and isolates the specific video segment corresponding to the detected anomaly for forensic processing.

List<Frame> extractPreEventFrames(): Retrieves the frames leading up to the incident to allow the system to analyze the root cause of the anomaly.

class TriggerLogic

Description: This class serves as the event-driven controller. It compares anomaly scores against pre-defined safety thresholds and manages the transition of the system state from "Observation" to "Active Diagnostic".

Attributes

int alertLevel: A priority ranking (e.g., Low, Medium, High) assigned to the current anomaly based on its score severity.

boolean isTriggered: A flag indicating whether the system is currently in an active diagnostic state.

Methods

void evaluateScore(float score): Compares the incoming anomaly score with global thresholds to decide if the diagnostic branch should be activated.

void activateDiagnosticBranch(): Sends an activation signal to the Diagnostic package and notifies the Global Control Service of the event.

class RegionMasker

Description: This class defines static areas of interest (AOI) within the camera's field of view. It filters out movement in non-critical zones (e.g., sidewalks or background trees) to reduce false alarms.
Attributes
List<Polygon> masks: A collection of geometric coordinates defining the boundaries of the active traffic lanes to be monitored.
Methods
boolean isInsideAOI(Coordinate vehiclePos): Checks if a specific vehicle's position falls within a defined monitoring zone before calculating its anomaly score.

3.3 Diagnostic and Forensic Identification Package Interfaces

class AccidentClassifier
Description: This class utilizes deep learning models to categorize the detected anomaly into specific incident types, such as multi-vehicle collisions, vehicle stalls, or illegal maneuvers.
Attributes
String modelType: The architecture identifier of the classification model currently in use (e.g., 3D-CNN or Vision Transformer).
float confidenceScore: The probability value associated with the identified incident category.
Methods

String classifyEvent(VideoClip eventClip): Analyzes the spatial and temporal relationships within the video clip to return the most likely accident category.

float getConfidence(): Returns the accuracy probability of the classification result.

class LPRManager

Description: This class provides the License Plate Recognition service. It isolates vehicle plate regions from high-resolution frames and extracts alphanumeric characters using OCR.

Attributes

String ocrEngine: The specific OCR engine configuration used for character extraction from identified plate regions.

Methods

Image detectPlate(Frame frame): Locates and crops the license plate area from the vehicle's bounding box in a specific frame.

String readPlateText(Image plateImage): Converts the cropped plate image into a digital string representing the vehicle's identity.

class FaultyVehicleIdentifier

Description: This class applies rule-based logic and trajectory analysis to determine which vehicle is responsible for the detected violation or accident.

Attributes

List<Rule> trafficRules: A set of encoded traffic regulations (e.g., lane discipline, speed limits) used to validate vehicle behavior.

Methods

int determineResponsibility(Tracks tracklets): Analyzes the pre-event trajectories to identify the ID of the vehicle that initiated the anomalous sequence.

boolean validateViolation(Event incident): Cross-references the incident context with the rule set to confirm if a legal violation occurred.

class EvidenceGenerator

Description: This class aggregates data from the classification and identification processes into a single, structured evidence package for legal and reporting purposes.

Attributes

JSON evidencePackage: A structured data container holding the incident type, faulty plate ID, and metadata links.

Methods

void compileEvidence(String type, String plate, int faultID): Combines disparate diagnostic findings into a unified record.

byte[] formatPackage(): Serializes the evidence data into a format suitable for encrypted storage and transmission.

3.4 Data Management and Reporting Package Interfaces

class DatabaseController
Description: This class manages interactions with the relational database (SQLite or PostgreSQL) to store incident metadata. It ensures that every detected event is indexed for rapid retrieval and forensic analysis.
Attributes
Connection dbConnection: The active session link to the local or remote database instance.
Methods
boolean insertIncident(JSON metadata): Writes structured incident details, including timestamps and classification types, into the database.
List<Record> queryHistory(Filter filter): Retrieves historical incident data based on specific search criteria such as date range or vehicle ID.

class EncryptionEngine
Description: This class implements security protocols to protect sensitive data. It handles the encryption of media clips and license plate information to comply with KVKK and GDPR privacy standards.
Attributes
byte[] secretKey: The AES-256 master key used for the encryption and decryption processes within the edge node.
Methods

`byte[] encryptFile(File rawFile)`: Converts raw media or text files into an encrypted format before physical storage.

`byte[] decryptData(byte[] encryptedData)`: Decodes stored information for authorized viewing through the client interface.

class AlertDispatcher

Description: This class acts as the system's communication hub. It is responsible for transmitting high-priority evidence and alerts to the remote dashboard via secure protocols.

Attributes

String `remoteEndpoint`: The URL or IP address of the central monitoring station's API.

Methods

`void sendAlert(byte[] evidence)`: Pushes the compiled and encrypted evidence package to the remote dashboard using HTTPS/TLS 1.3.

`boolean checkConnection()`: Periodically verifies the network status to ensure the alert transmission path is active.

class StorageJanitor

Description: This class manages the local storage lifecycle on the NVMe SSD. It implements automated retention policies to balance disk space availability with data preservation requirements.

Attributes
int retentionPeriod: The number of days an incident record is kept before being flagged for archival or deletion.
Methods
float monitorDiskSpace(): Checks the current storage utilization level and triggers cleanup if thresholds are exceeded.
void purgeOldLogs(): Automatically deletes non-critical logs and old footage that has surpassed the retention period, while preserving flagged evidence.

4. GLOSSARY

AES-256: Advanced Encryption Standard with a 256-bit key; a symmetric encryption algorithm used to secure media files and sensitive metadata.

Anomaly Trigger: A signal generated by the Anomaly Analysis package when a vehicle's behavior exceeds a predefined statistical threshold, initiating the reactive diagnostic branch.

Bounding Box: A rectangular border that encloses a detected object (vehicle) in an image, defined by (x, y) coordinates, width, and height.

ByteTrack: A high-performance multi-object tracking algorithm that associates detection boxes across frames by leveraging low-score detections to maintain track continuity.

Circular Buffer: A data structure that uses a single, fixed-size buffer as if it were connected end-to-end; used for temporary storage of the most recent video frames.

Edge Node: The specialized hardware platform equipped with dedicated AI accelerators where the ADVIS pipeline is executed in real-time.

Forensic Identification: The process of performing deep, post-event analysis to identify license plates and determine fault after an anomaly is detected.

LPR (License Plate Recognition): A technology that uses optical character recognition on images to read vehicle registration plates.

Main Execution Loop: The continuous processing cycle in the Perception package that remains active to monitor traffic regardless of whether an accident has occurred.

Metadata: Structured data that provides information about other data; in ADVIS, this refers to vehicle IDs, coordinates, and speed vectors.

RTSP (Real-Time Streaming Protocol): A network control protocol designed for controlling streaming media servers, used to ingest video from traffic cameras.

Tracklet: A fragment of a vehicle's trajectory consisting of a series of associated bounding boxes over a specific time window.

YOLO (You Only Look Once): A real-time object detection neural network used in the perception layer to identify vehicle classes and position

5. References

ISO/IEC 19505:2012: Information technology — Object Management Group Unified Modeling Language (OMG UML) — Part 1: Infrastructure.

IEEE Standard 1016-2009: IEEE Standard for Information Technology—Systems Design—Software Design Descriptions.

Redmon, J., & Farhadi, A. (2018): "YOLOv3: An Incremental Improvement." *arXiv preprint arXiv:1804.02767* (Reference for the Object Detection Service).

Zhang, Y., Sun, P., et al. (2022): "ByteTrack: Multi-Object Tracking by Associating Every Detection Box." *Proceedings of the European Conference on Computer Vision (ECCV)* (Reference for the ID Management Service).

Official GDPR Documentation: Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data.

6698 Sayılı Kişisel Verilerin Korunması Kanunu (KVKK): Republic of Türkiye Official Gazette, No. 29677 (Reference for security and encryption standards).

ADVIS High-Level Design (HLD) Report: The foundational architectural document providing subsystem decomposition and system requirements.